Dataset 1: Final Vial Inspection

Authors: MSOE AI Club

Published: October 2026

Inspect final vial fixtures for missing or crooked caps, damaged vials, and unexpected empty positions using real and synthetic factory images.

Dataset 1: Final Vial Inspection

Back to the Innovation Lab overview

The problem

Vials leave the production line in two fixtures at the final station, one for good vials and one for scrap. Build a system that uses an overhead camera at that station to check visible vial defects and fixture occupancy after each placement. The priority is to stop defects from getting through. In judging, a missed defect matters more than a false alarm.

CSI HACKATHON CHALLENGE

Final Vial Inspection

Train an automated system to detect visible vial defects and check fixture occupancy after the robot places each vial.



Image 30: the good fixture is on the left, scrap on the right. Each fixture has 30 fixed positions. The camera observes the vials after placement.

Why another inspection? The upstream station views vials from the side and routes them to good or scrap. It may miss cap defects. During handling, the robot can miss a pick, damage a vial, or dislodge a cap without vision feedback.

Completion and gaps. A good order is visibly complete when all 30 positions contain acceptable vials. An internal gap can indicate a missed placement. Trailing empty positions may be waiting for vials; detecting a missed final placement requires a placement count or completion signal.

The production line already inspects each vial from the side. It sends vials that pass to a good fixture and detected defects to a scrap fixture. That inspection does not see every cap defect. A cap can also move after inspection, and the robot can miss a pick, damage a vial, or place it incorrectly without noticing.



How the fixtures fill

The camera sees two fixtures:

- **G** is the good fixture.
- **S** is the scrap fixture.
- Each fixture has 30 fixed, numbered positions.
- The robot fills the rightmost column from top to bottom, then moves one column to the left.

waiting for later vials, so a robust solution should use placement count or an order-complete signal when one is available.

Labels

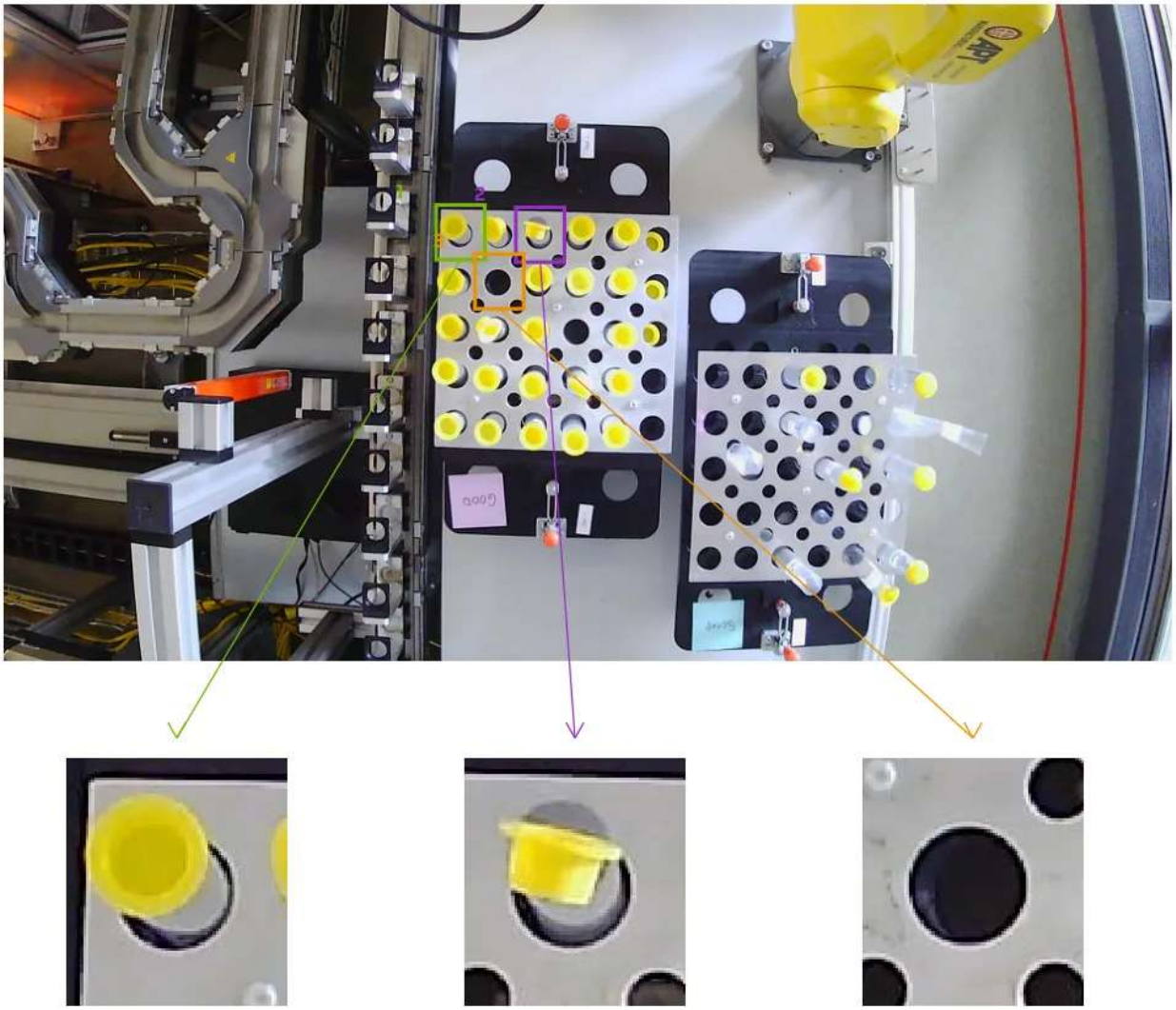
The class label describes one target position, not the whole camera image. Crops contain neighboring vials, so make sure the prediction follows the numbered target aperture.

Label	Meaning
`pass`	The visible cap and vial body appear acceptable. It does not confirm properties hidden from the camera.
`missing_cap`	A vial is present, but its mouth is open. Do not confuse the transparent vial body with an empty fixture hole.
`crooked_cap`	The cap is visibly tilted or not seated. A leaning vial by itself is not enough.
`damaged_vial`	The target vial body is visibly damaged. Overlapping transparent bodies may require the full scene for context.
`no_vial`	The target aperture is empty. Whether that is an error depends on fill progress.

IMAGES AND LABELED CROPS

One image, many inspection targets

A full camera image shows both fixtures and the surrounding equipment. Each class label belongs to a crop around one numbered fixture position, not to the entire image.



1. G26: pass

2. G16: crooked_cap

3. G22: no_vial

Image 126. Colored boxes link the full image to three enlarged crops. Each crop includes context and may show neighboring vials. Its label describes the vial, or empty aperture, at the target position. Each fixture has 30 numbered positions.

LABELED CROP EXAMPLES

What the system should recognize

Each example below is a labeled crop from a larger camera image. The green ring marks the target position. Crops include neighboring vials; follow the target aperture.



Missing cap / missing_cap

A vial is present with an open mouth. Distinguish its transparent body from an empty fixture hole.

Crop from image 1, position S01



Crooked cap / crooked_cap

The cap is visibly tilted or improperly seated. Inspect cap seating; vial lean alone does not establish this class.

Crop from image 126, position G16



Damaged vial / damaged_vial

The target body is labeled damaged. Transparent bodies overlap here, so inspect the target body in the full scene as well.

Crop from image 1, position S08



Empty position / no_vial

The target aperture has no vial. This describes occupancy; whether it is a missed placement depends on fill progress.

Crop from image 126, position G22



Passing example / pass

The visible cap and body appear acceptable. This label does not establish fill level or other properties hidden from the camera.

Crop from image 126, position G26

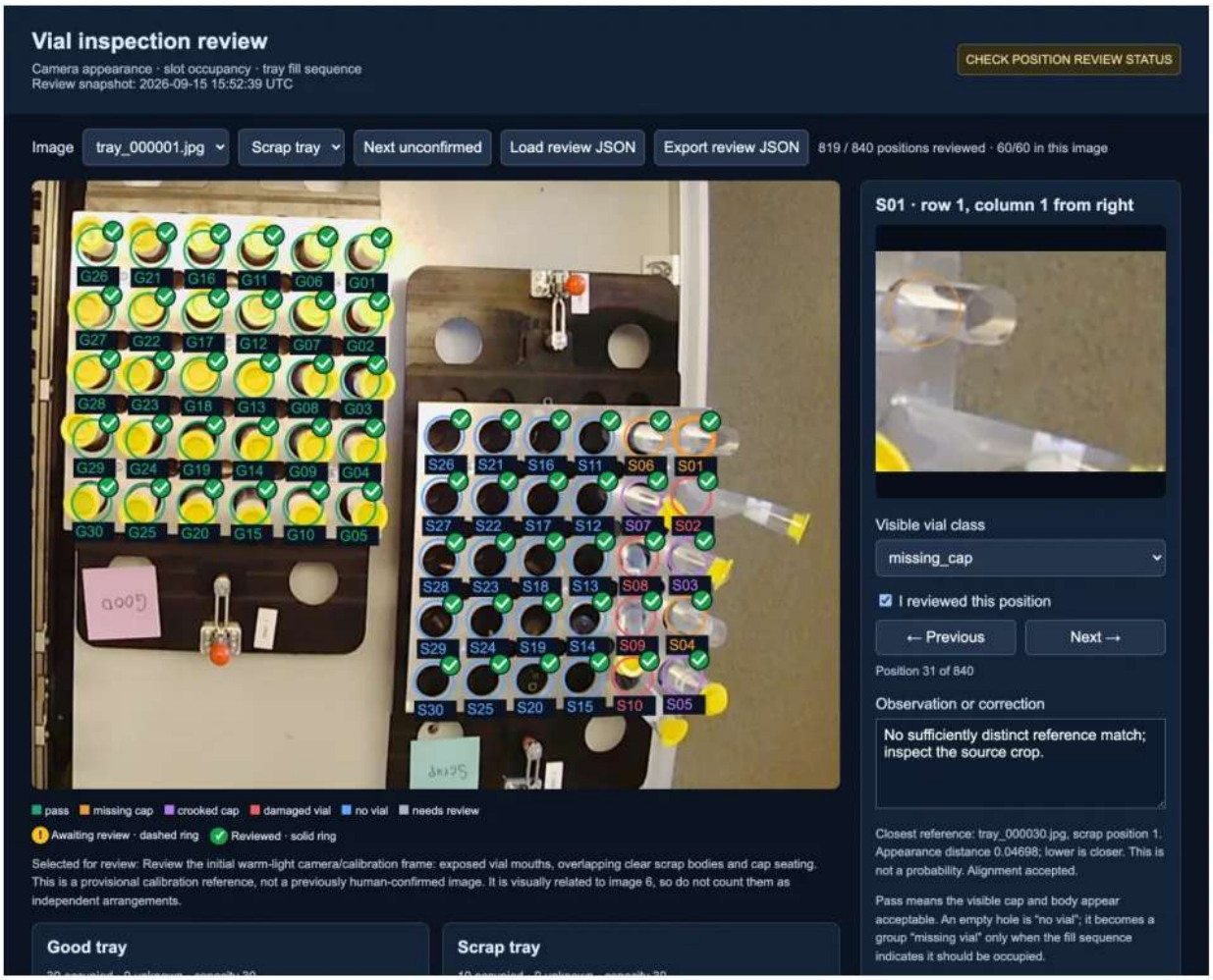
- Sample data from the real line, from the overhead camera at the final station:
  - full camera images of both fixtures;
  - labeled and unlabeled crops of each numbered position, using the five labels above;
  - fixture coordinates and position identifiers, so every crop maps back to its source image, fixture, and position.
- A browser-based labeling tool for reviewing and labeling positions.
- A digital twin for generating training images and labels.

The labeling tool lets you select an image and fixture, click a numbered position, inspect its crop in scene context, edit the class, add an observation, and mark it reviewed. Use Previous and Next to move through positions. Export the review JSON when the pass is complete.

LABELING TOOL

Review a crop in its scene context

The repository includes a browser-based labeling tool. Select an image and fixture, then click a numbered position to view its crop and edit the class.



Tool screenshot: image 1, scrap position S01, labeled missing_cap. The scene is on the left; the selected crop and editing controls are on the right.

Review and save. Inspect the target aperture, choose the visible vial class, add an observation if needed, and mark the position reviewed. Use Previous and Next to move through positions, then export the review JSON.

Digital twin (datasets 1 and 2): Download the digital twin

Dataset and labeling tool: the download will be linked in `#dataset-1-vial-inspection` on Slack.

Everything in one place: Innovation Lab data folder

For MSOE students on Rosie: use the download link above until an on-Rosie path is published.

Look at the real data, but train on the digital twin

The sample images come from the real line. Look through them to understand the station, the defects, and what your generated images need to match, but **avoid training your models on them**. The goal of this lab is to generate your training data with the digital twin.

That is how this problem has to be solved on a high-mix, low-volume (HMLV) line like CSI's, which makes many product variants in small batches and changes over often. A new product or recipe has no real images until the line has been commissioned and is running it. Collecting and labeling real examples afterwards is costly: it takes line time and operators, defects have to be produced on purpose, and the work repeats at every changeover. A digital twin can produce labeled images of a product, including rare defects, before it ever runs on the line.

Need more real data? See Collect and Test Your Own Data. The dataset and labeling tool will be linked in `#dataset-1-vial-inspection` on Slack.

The Isaac Sim on Brev tutorial shows how to run a digital twin headless and copy rendered images back to your computer.

Starter approaches

These are ideas to get you started, not a required plan. Combine them, change them, or take a different direction.

defects that are rare in the real data, and its images come with labels.

- **Train a lightweight inspection model.** Visual ChangeNet in the TAO Toolkit compares each position with a known-good reference and outputs `PASS` or `NO_PASS`. The example below walks through it.
- **Reason with Cosmos Reason.** A vision-language model can inspect a crop from example images in its prompt, without first training a small dedicated model.
- **Use the fill pattern.** The robot fills positions in a fixed order, so an empty position inside the pattern can mean a missed placement, while empty positions at the end may just be waiting for later vials. Use the placement count or an order-complete signal when one is available.
- **Iterate on the cases it gets wrong.** The DEFT AOI workflow is an evaluate, diagnose, augment, retrain, and gate loop.

Example: TAO Toolkit with Visual ChangeNet

One strong baseline is a small Visual ChangeNet classification model trained on images you generate with the digital twin. Visual ChangeNet compares each inspection image with a **golden image**, a known-good reference of what the vial should look like, and outputs `PASS` or `NO_PASS`.

The general approach:

- **Compute:** if you use Brev, prefer an RTX PRO 6000 when available; its higher hourly rate may be offset by faster training. An L40S on an AWS environment remains a capable lower-cost fallback. See the Compute guide.
- **Generate your data:** render good vials and each defect class from the digital twin, matching the real station's overhead camera pose, crop, and lighting. Visual ChangeNet trains on pairs of an inspection image and a golden image in the TAO annotation format.
- **Pick a golden image:** choose one confirmed-good vial crop and point your agent to it. It should match the inspection images' position, camera pose, crop, scale, and lighting, or the model may learn to flag imaging differences instead of defects. Position-specific or multiple golden images are worth exploring later.
- **Train with an agent:** clone the NVIDIA TAO Skill Bank and point your coding agent at its Visual ChangeNet skill. The Skill Bank gives the agent model-specific instructions, data checks, and container steps, so you do not need to memorize TAO commands. Follow its README for setup; the TAO container and backbone may require your own NGC and Hugging Face accounts.

When you read the results:

- **Positive means a defect was detected.** A false negative is an escaped defect, and a false positive is a good vial rejected. Missed defects matter more in judging.
- The model's score is a distance from the golden image, not a probability. Higher scores lean toward `NO_PASS`.
- Report the false alarm rate at the threshold that catches every known defect, not only overall accuracy.
- Open several correct and incorrect image pairs to check whether the model reacts to the defect or to pose, glare, and lighting.
- Change one thing at a time and keep the test set fixed.

Questions that may help narrow the first experiment:

- Is it easier to classify a fixed crop for each position, or detect all visible vials in the full image?
- How will the system separate `no_vial` from a transparent uncapped vial?
- What threshold gives the right tradeoff when missed defects cost more than false alarms?
- How will you avoid reporting trailing empty positions as defects before the order is complete?
- Which defects are underrepresented, and can the digital twin create useful examples?

Relevant tools include the TAO Toolkit, Visual ChangeNet, the DEFT AOI workflow, Cosmos Reason for in-context inspection, and Isaac Sim with Replicator for synthetic images and labels from the digital twin. See Tools and Resources, and see Compute before launching training or simulation.

Testing

Keep a test set that is not used for prompt examples, augmentation decisions, or training. Report results per class, not only overall accuracy.

Report false alarms and missed defects separately, and show the false negatives for each defect class. Test at least a few complete fixture scenes, not only individual crops.

For additional collection or live-line testing, follow Collect and Test Your Own Data.